

FileShare - P2P Application

Due Dates/Milestones:

- Semester-Start Check-In: 23/07/2026
- Milestone 1 Check-In: 05/08/2026
- Milestone 2 Check-In: 19/08/2026
- Expected Completion: 26/08/2026

You (and your group) are now tasked with writing an application that will handle peer connections, share/send file data on request, handle different connection events gracefully, report file download progression. This application will have more than one component that will be developed.

This assessment can be performed in groups of 1, 2, or 3. You will have been sorted into group prior to the release of the assessment. Depending on what group composition you are part of, the group will need to address the tasks that have been outlined.

Background and Base Scope

Straight after building the merkle-tree, you are to build the rest of the application. As described prior, this application is to share files between peers and validate that the files and data-blocks transferred are written to the correct location and match the expected hash.

While this application main focus is serve as a peer-to-peer file-sharing application, it will also need a number of features to support this feature. These will include:

- Sharing multiple files
- Connection to multiple peers
- Querying and managing connections between peers
- Querying and managing the file sharing states
- Web Interface for viewing and managing, connections, files, states, and configuration
- Database to manage current file-sharing state
- Interacting with the filesystem to read and load files
- Implementing the network protocol called “simple-share”
- Error handling on the relevant components
- Constructing and utilising configuration file for web-server port, socket listening port and file directory and metadata file directory
- Compatibility with other implementations (Your software has to work with other groups projects)
- User-Documentation on the Web client

The codebase is to built primarily using C# and ASP.NET along with TCP/IP Network Sockets and EntityFramework/ADO.NET.

Previous Project

This assessment requires Assignment 2 - Part 2 to be completed or in a reasonable state. The assessment is a natural extension of the previous one as it builds on top of the already constructed library.

However!, the usage of the library should be used only as a **library** and not as anything more than that. You and your group will need to adhere to a number of constraints when using this library.

- FileValidate class-library can only be used as a library and is to be separate from the project.
- FileValidate must be in a separate repository.
- Bugfixes can be submitted and applied to the FileValidate library.
- Features and Improvements are permitted but must be rationalised.
- Any changes should be documented via an issue tracker (example: GitHub issues).

While contributions are accepted, the first priority is building a compatibility layer to also enable the library to be swappable. When working with a group, consider swapping the **FileValidate** library and using it between versions as a way to test your abstractions.

Development

Regardless if you are working on this project by yourself or in a group, you will need to have your software hosted on a **git** repository. You may use **Github**, **Gitlab**, **Codeberg**. Please specify what service you are using with your supervisor so they are able to be added.

Additional Scope (Duos and Trios)

As a group of size 2, you will be given additional tasks to handle.

- **.tpk** File Maker, allows the user to create a **.tpk** file that can be used with your application.
- Issue Tracking with the elected platform
 - Clear assignment of issues to group members.
 - Issues should be tracking: Features, Improvements, Bug-Fixes and other tasks
 - Group should be meeting at least once weekly and confirm what tasks each person is doing.
- Automated Actions for running tests against your software when a commit occurs
- Documentation of the code-base, make sure you can generate the documentation from the **C#** build system.

Additional Scope (Trios)

As a group of size 3, you will be given additional tasks to handle as well as the duos scope.

- Automated Build System, Compilation, and Distribution: Building for multiple platforms - Targeting Windows and Linux
- Text-User Interface Client that can interact with the daemon.
- User-Documentation on the TUI client
- Communication to the daemon process via a named pipe

Project Components, Artifacts, and Requirements

Given the scope, features and methodology you will need to maintain. You will need to adhere to break your project up into components which will be worked on separately, along with deliverable artifacts and adhere to the requirements in the sections below.

Components

The project is separated in the following components.

- **FileShare.Web** - This component relates to the web interface (and service) that the will handle connections from peers, file states, user interface and other core activities.
- **FileShare.Lib** - This component relates to a shared set of logic that would be used amongst the rest of the projects and allows the project to be tested in isolation.
- **FileShare.Test** - xunit testing project that will require the user to test their code against **FileShare.Lib** and demonstrate that it is working as expected.
- **FileShare.Service** (Group(3) Only) - The service component is the **daemon** that will run in the background and handle connections from peers and file-state. (Only groups of size 3 are required to build this - It separates the logic that would otherwise exist in **.Web** component).
- **FileShare.Console** (Group(3) Only) - This component is the TUI application that interacts with the **.Service** instance via a **Named Pipe** (Only groups of size 3 are required to build this).

The above projects are simply scaffolds and will require you to craft your own solution.

Artifacts

Artifacts are deliverables objects that are expected. The artifacts that are expected from this project are the following.

- **FileShare-<platform>** - Application that is built for Windows and Linux, you should have different versions for the different build systems. You should be delivering two executables that must run on `x86_64` builds of these systems.
- **FileShare-Documentation** - Documentation from the codebase should be generated from XML comments inside the codebase. Use `DocFX` or `Sandcastle` to generate the documentation.
- **FileShare-Coverage.report** (Group(2) and Above) - A code coverage report should be generated to outline what methods have a test case associated and what does not.

Requirements

Given the scope in the previous section, use the category for guidance on feature implementation.

1. Loading and Configuring Files Your application must parse and load a configuration file that will be used as part of initialising folders that it will either need to create or, if it exists, load existing packages from and refer to an existing file.

The program's configuration file will use the following information:

- **metafile_directory**, string, path local to the application and can be either an absolute path or relative path.

Used to load, copy, and store `.tpk` files. **FileShare** application must be able to read the directory and load the files.

If the directory does not exist, the program should attempt to create it by creating a folder where it has been specified.

If the program is unable to create the directory or it is a file, the program should exit and output to `stderr` the message: `"Unable to create metafile_directory"`.

- **file_directory**, string, path local to the system that store `.tpk` files and the files that are mapped in there. This directory can be either an absolute path or relative path. Given the `.tpk` files should have existing files in this location, the application must be able to load existing files and understand their state.
 - If the directory does not exist, the program should attempt to create it by creating a folder where it has been specified.
 - If the program is unable to create the directory or it is a file, the program should exit and output to `stderr` the message: `"Unable to create file_directory"`.

- **max_peers**, int, this field is the number of peers the program can be connected to. It is a value between 1 and 2048.
 - If the **max_peers** value is set to an invalid number, your program should exit and output to **stderr** the message: **"max_peers within the configuration file is set to an invalid value"**
- **port**, int, this field specifies the port the client will be listening on. Make sure that when your application is listening for connections (not web-server connections, just socket connections), it is listening to the port specified.
 - If the **port** value is set to an invalid number or the port is in use by another application, your application should output to **stderr** the message: **"Port specified is either in use or invlaid"**
- **nickname**, string, this field specifies the nickname of the peer participating on the network. This does not need to be a unique **nickname** as the ip and port also make up part of the identification.

Example of the configuration file:

```
nickname=peer-iso
metafile_directory=descriptors
file_directory=downloads
max_peers=128
port=9000
```

Each field has an action if the constraints of that field are not met as above. If any fields are missing, the configuration should be rejected.

2. Connection Protocol Your program can act as both a server and client to participants among the network. You will need to be able to form a listening socket to accept incoming connections but also have the ability to form new connections.

The network protocol will use TCP/IP data messages to form connections. Your program should use the following message structure below.

```
message {
    msgcode    : unsigned short
    msglength  : unsigned short
    payload    : unsigned byte[msglength]
}
```

The maximum size of the message is **4096** bytes, however, messages will be between **4 bytes** or **4096 bytes**.

The following message codes (**msgcode**) are supported in the core protocol. Each **message**

- **ACP (0x02)**, when a peer connects to your program, you will need to acknowledge that you have accepted the connection so it can confirm it can add it to its own peer list.

If your program connects to peer, you should wait for an **ACP** message back before you add the peer to your own peer list.

If the peer does not respond within a *reasonable* time, your program can kill the connection.
- **ACK (0x0c)**, when your program receives an **ACP** message after a connect, you will send a message back with an **ACK**. This is to simply acknowledge that you have received the message.
- **IDA (0x01)**, when your program receives an **IDA** message, this is a peer attempting to gain a nickname of the peer. Your program should simply respond with the identifier that was specified within the configuration file, please see **IDR** message.
- **IDR (0x02)**, when your program receives an **IDR** message, this is a response message from an **IDA** message. It will contain a string under the **payload**. The largest nickname is **512** characters. Make sure you check for a mismatch on the message length and actual message sent.
- **DSN (0x03)**, when a peer wants to disconnect from another peer, it will send a message to the peer, telling that it will be disconnecting. The peer that originated the message will close off its socket and end the connection.
- **ERR (0x04)**, when a peer has sent a message that cannot be handled by our client, this message type is designed to inform the peer that they have sent an invalid message.

- **REQ (0x06)**, This message is sent to a peer to request data for a particular chunk. The request message will send both the **<identifier>** (1024 bytes), **<chunk index>** (4 bytes, **uint32**) and **<offset>** (4 bytes, **uint32**) to another peer in the expectation that the peer will send data back.

<req len> must have a value which is larger than 0 and less than the chunk size. If the requested length is not a valid value when received, the request is ignored.

If the peer sends you a **REQ** message but you do not have the expected file or chunk available, you will need to inform the requester of an error in the **RES** message.

- **RES (0x07)**, This message is sent to an originator of a **REQ** message, it will contain: **<identifier>** (1024 bytes), **<chunk index>** (4 bytes, **uint32**), **<offset>** (4 bytes, **uint32**), **<data len>** (2 bytes, **uint16**) and most importantly **<data>** (max 3054 bytes).

The response message will send data from the chunk to the requester, since the file **<data len>** component may not match the **<req len>** component of a **REQ** request message, the peer will need to send multiple **RES** messages to satisfy the length of data requested.

The **<offset>** data is used to tell the part of the chunk to start sending from, this is useful information for when the peer wants.

If the peer does not have the data available, an error byte fields should be set to a number > 0 . This will notify the requester that the current peer does not have the data requested.

The following message, can be sent multiple times, this is normal as part the **REQ-RES** flow, a peer will normally send a single **REQ** message and receive N **RES** messages to satisfy the chunk.

- **PNG (0xFF)**, This message is sent out with the intent to check if a peer is still alive. Nominally, this is usually sent out periodically,

No error handling is necessary for this particular message.

- **POG (0x00)**, This is a pong message that will be sent to the originator of the **PNG(0xFF)** message.

3. User Interface Features You will need to implement the following forms and components. Your web user interface will implement success and error handling mechanism for actions and states.

- **Connect To Peer** - Attempt to connect to a peer within the network itself. Your program will need to construct a socket and attempt to connect to another peer on the network.

The form will display the IP and PORT that the server will connect on.

If the ip and port given, already have been connected to and the connection is alive, your application should output a message to the user. **Already connected to peer**

If a the ip or port has not been specified, your application should output to the user. **Missing address and port argument.**

For the web interface, this should be a button, typically on the top-bar of the application. For the text-user interface, this will be in the form of a **CONNECT <ip:port>** command.

- **Disconnect** - will disconnect from a peer but not remove it from a peer list. Please refer to the DSN message in the network protocol section.

If the peer is connected and in your program's peer list, your program must disconnect with the peer and output **Disconnected from peer.**

If the peer does not exist in your program's peer list, your program must output: **Unknown peer, not connected**

If a the ip or port has not been specified, your program should output **Missing address and port argument.**

For the web interface, this should be a button associated with each user. For the text-user interface, this will be in the form of a **DISCONNECT <ip:port>** command.

- **Add Package** - will add a package to manage.

If a the file has not been specified, your application should output: **Missing file argument.**

If the file does not exist or you do not have permission to use it, Your application must output: **Cannot open file**

If the file is not a valid tpk file, your application must output **Unable to parse tpk file.**

For the web interface, this should be a button, typically on the top-bar of the application, next to **Connect To Peer**. For the text-user interface, this will be in the form of a **ADDPACKAGE <file>** command.

- **Remove Package** - will remove a package that is being maintained by the application.

On success, the program will output `Package has been removed`

Within the web interface, each loaded package should have the ability or a button to be removed, this should prompt the user

Within the text user interface, the command will be in the form: `REMPACKAGE <ident: identifier or partial ident:, 20characters matching>`.

If the `ident` is not managed by your program, your application should output: `Identifier provided does not match managed packages`

If a the `ident` has not been specified, your application should output `Missing identifier argument, please specify whole 1024 character or at least 20characters`.

- **Packages** - Displaying a list of packages that are currently being operated on.

Your program should report on the status of the packages loaded. Your program will have need to maintain a list of packages that have been added in working memory.

If your program is not managing any packages, your program will display `No packages managed`.

If your program is managing 1 or more packages, your program will output in the following format for the text user interface:

`[1..N]. <32 characters of the identifier>, <filename> : (INCOMPLETE | COMPLETED)`

Example:

1. `0c4d036a2161aa6525743d44725e6212, song5.mp3 : INCOMPLETE`
2. `13d608773eb8842426fddb8131d5c184, song6.ogg : COMPLETED`

For the web user interface, the display will show a container for the following.

The following is a description of the components of a package entry. This is to be listed in the **Context** UI.

- Each entry is part of a table or list.
- Either the full identifier or a cut-off version
- Current state of the file - If it is complete or incomplete
- The progress of the file as a percentage (number of blocks completed)
- **Peers** - Lists all the connected peers.

Within the text user interface.

If your program is not connected to any peers, it will display `Not connected to any peers`

If your program is connected to 1 or more peers, your program will output in the following format:

Connected to:

[1..N]. <ip>:<port>

Example:

Connected to:

1. 192.168.1.1:9001

2. 192.168.2.120:1723

...

Within the web interface, you will display your peers within the **Context** component.

If your program is not connected to any peers, it will display the following in the context component. **Not connected to any peers**

When connected to one or more peers, it will display the each peer within the **Context** component. Each peer will be displayed as described.

- Each entry is part of a table or list.
- Nickname of the identifier, if a nickname cannot be specified, the peer can be categorised as **Unknown#ID** where **#ID** is a unique number in the sequence from 1 to infinity.
- IP Address and Port Number that you are connected to with the peer.
- If the peer is still connected or disconnected - A button to **Reconnect** with the peer should be present.

4. Web User Interface Use the following wireframes as a guide for organising information to be displayed. These wireframes are to simply provide a starting point and can be extended.



Figure 1: Components



Figure 2: Simple Packages View

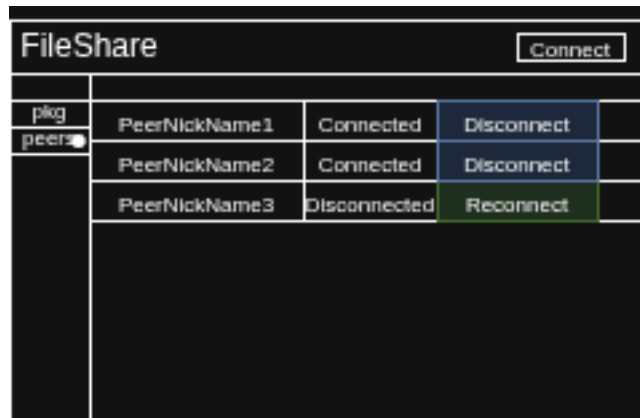


Figure 3: Simple Peers View

Instructor - Hints and Tips

Since you are building upon another application and starting with what is akin to a blank slate for your project. There are some hints and tips I would like to outline here.

- Use effective data structures here - Maps and Lists are very powerful for maintaining your objects as well as a database when you want persistence.
- Think about what objects are stateful and what are stateless. Stateful objects are going to have different behaviour depending on what state they are in and consider the kind of design pattern you may use here.
- Creating wrappers and abstractions that you can switch and experiment with gives you freedom in your design and choices.
- Use the class library! Separating the logic from the UI allows the core logic to be tested easily. When they are tightly coupled, it can make the code quite fragile and any modifications can have significant flow-on effects.
- Have roles but also swap roles! As part of building up your experience with these tools, you want to
- Utilise issue tracking! Keeping track of issues that you and your group have allows you think more about the problems than trying to think what the problems are. It also creates a separation of what features, bugfixes and improvements that are being undertaken by others.
- You will be using `git` in a more involved way. Please create `feature/`, `bugfix/` and `improvement/` branches with your repository. These branches can be related to an issue and tracked as well.
- Meet-up at least once a week - During semester, your instructor will be

your project manager and ensure that all members of the group have at least 3 tasks for the next week. During semester and term breaks, use the time to get a head-start with a weekly meetup on what tasks you are working on.

- Keep-each other Accountable, Be Compassionate, Understand when and why a task wasn't completed. This is to appeal to developing a responsible attitude towards software development.
 - Tasks can be late - Potentially the task was more than what was perceived
 - * Tasks might not be doable - Understand why it wasn't completed and consider how to address it
 - * Tasks might be ill-defined - It is difficult to get the perfect definition of what **Complete** means. Consider what the definitions is for each task and what constitutes **Complete**
 - * Test cases provide accountability and avoid arguments - A team member can say "It is done" but their definition may be different from others or the conditions in which it was executed were misleading. This miscommunication is common and it is difficult for others to understand if something is complete without going through all the steps necessary in the app to check it, in which mistakes can occur. Test cases help address this by providing a way to demonstrate a completed task that can be repeated by others.
 - * Miscommunication Happens - We all have slip-ups with what we are referring to and it is always best to consider best intentions, even if it is difficult to see it.

Assumptions

This assessment makes a few reasonable assumptions around how messages and data will be encoded without explicitly outlining it in each section. It is also reasonable to make it clear how communication is to be afforded if between all peers.

For sake of simplicity, file and network binary data is sent or saved as little endian. For singular bytes this does not have any serious bearings however, for integers this is significant to outline as the order of bytes may be different between a BE and LE system with this software.

Submission

You will need to ensure that your submission is up to date and the **main** branch contains your final submission. It is recommended you use **Codeberg** to host your project due to changes to Github.

Your final submission must include:

- URL and access to the repository with an accessible **main** branch
- A buildable project on Linux and Windows that can be invoke by **dotnet build -c Release**.
- A clear commit history along with clear statements of student contribution and roles they undertook.
- Group associated artifacts that are required part of the tasks.

If you run into any issues while working on the project, please reach out to your instructor.

Good Luck and Happy Coding!